Lineáris egyenletrendszerek

numerikus megoldása

Gauss-elimináció · LU · Cholesky

Jacobi · Gauss–Seidel

2026. október 5.

Tartalom

1. A feladat
2. Gauss-elimináció
3. Pivotálás
4. Kondicionáltság
5. LU-felbontás
6. PLU és Cholesky
7. Iteratív módszerek
8. Jacobi-iteráció
9. Gauss–Seidel-iteráció
10. Konvergencia

A feladat

Keressük az $Ax = b$ egyenletrendszer megoldását,

ahol $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$.

Miért numerikusan?

- A Cramer-szabály $n!$ nagyságrendű művelet — $n = 20$-ra több, mint 10^{18}.
- Az A^{-1} kiszámítása drágább *és* pontatlanabb, mint az egyenletrendszer közvetlen megoldása.

Kétféle megközelítés

Direkt módszerek

- véges sok lépés
- kerekítéstől eltekintve pontos
- $\approx n^3/3$ művelet
- Gauss, LU, Cholesky

Iteratív módszerek

- közelítő sorozat
- soha nem pontos
- lépésenként $\approx 2n^2$
- Jacobi, Gauss–Seidel

Nagy, ritka mátrixnál ($n \sim 10^6$) a direkt módszer kivitelezhetetlen.

Példa

$$
\begin{aligned}
-2x_1 - x_2 + 4x_3 &= 3 \\
2x_1 + 3x_2 - x_3 &= 1 \\
-4x_1 - 10x_2 - 5x_3 &= -12
\end{aligned}
$$

$Ax = b$, ahol

$$
A = \begin{pmatrix} -2 & -1 & 4 \\ 2 & 3 & -1 \\ -4 & -10 & -5 \end{pmatrix}, \qquad b = \begin{pmatrix} 3 \\ 1 \\ -12 \end{pmatrix}
$$

Gauss-elimináció

$$\left(\begin{array}{rrr|r} -2 & -1 & 4 & 3 \\ 2 & 3 & -1 & 1 \\ -4 & -10 & -5 & -12 \end{array}\right) \xrightarrow[\ell_{31}=2]{\ell_{21}=-1} \left(\begin{array}{rrr|r} -2 & -1 & 4 & 3 \\ 0 & 2 & 3 & 4 \\ 0 & -8 & -13 & -18 \end{array}\right)$$

$$\xrightarrow{\ell_{32}=-4} \left(\begin{array}{rrr|r} -2 & -1 & 4 & 3 \\ 0 & 2 & 3 & 4 \\ 0 & 0 & -1 & -2 \end{array}\right)$$

ℓ_{ik} = az a szám, amellyel a k. sort szorozva kivonjuk az i. sorból.

A visszahelyettesítés

$$\left(\begin{array}{ccc|c} -2 & -1 & 4 & 3 \\ 0 & 2 & 3 & 4 \\ 0 & 0 & -1 & -2 \end{array}\right)$$

$$\begin{aligned} -x_3 &= -2 \quad \rightarrow \quad x_3 = 2 \\ 2x_2 + 3x_3 &= 4 \quad \rightarrow \quad x_2 = -1 \\ -2x_1 - x_2 + 4x_3 &= 3 \quad \rightarrow \quad x_1 = 3 \end{aligned}$$

Alulról felfelé haladunk — minden lépésben egy ismeretlen marad.

Általános alak

A kibővített mátrix, és az 1. lépés eredménye ($a_{11} \neq 0$, $\ell_{i1} = a_{i1}/a_{11}$):

$$\left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & & & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array}\right) \longrightarrow \left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ 0 & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ \vdots & \vdots & & \vdots & \vdots \\ 0 & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} & b_n^{(2)} \end{array}\right)$$

Az első oszlopban a főátló alatt mindenhol nulla áll.
Ezt ismételjük a 2., 3., ... oszlopra.

A k-adik lépés képletei

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - \ell_{ik}\, a_{kj}^{(k)}, \qquad \ell_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}$$

$$b_i^{(k+1)} = b_i^{(k)} - \ell_{ik}\, b_k^{(k)}$$

$$i = k+1, \ldots, n, \qquad j = k+1, \ldots, n$$

Feltétel: $a_{kk}^{(k)} \neq 0$ minden lépésben.

Mikor hajtható végre?

A sorcsere nélküli Gauss-elimináció pontosan akkor hajtható végre, ha

$$a_{11}^{(1)} \neq 0, \quad a_{22}^{(2)} \neq 0, \quad \ldots, \quad a_{nn}^{(n)} \neq 0.$$

Ha valamelyik nulla, akkor sorcserére van szükség $\Rightarrow$ pivotálás.

Műveletigény

1 művelet := 1 összeadás + 1 szorzás

Az A mátrix átalakításához (1. lépés: $(n-1)^2$, 2. lépés: $(n-2)^2$, ...):

$$(n-1)^2 + (n-2)^2 + \cdots + 1 = \frac{n(n-1)(2n-1)}{6} = \frac{n^3}{3} + \mathcal{O}(n^2)$$

A b vektorhoz: $\frac{n(n-1)}{2}$ A visszahelyettesítéshez: $\frac{n(n+1)}{2}$

Az $n^3/3$ tag dominál $\Longrightarrow$ a költség köbös.

Miért kell pivotálni?

Példa

Oldjuk meg 10-es alapú, 16 jegyű aritmetikában, pivotálás nélkül és pivotálással is:

$$\begin{aligned} 10^{-17}x \;+\; y &= 1 \\ x \;+\; y &= 2 \end{aligned}$$

Elméleti megoldás: $x = \dfrac{1}{1-10^{-17}},\ y = 1 - \dfrac{10^{-17}}{1-10^{-17}}$

Mindkettő kerekített értéke $x \approx 1,\ y \approx 1$.

Pivotálás nélkül

Az elimináció után ($\ell_{21} = 10^{17}$):

$$\begin{pmatrix} 10^{-17} & 1 \\ 0 & \mathrm{fl}(1 - 10^{17}) \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ \mathrm{fl}(2 - 10^{17}) \end{pmatrix}$$

A kerekítés után ténylegesen ábrázolt rendszer:

$$\begin{pmatrix} 10^{-17} & 1 \\ 0 & -10^{17} \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ -10^{17} \end{pmatrix}$$

Megoldása: $y = 1$, $x = 0$ — katasztrofálisan rossz!

Pivotálással

Sorcsere után a nagyobb abszolút értékű elem kerül a főátlóba:

$$\begin{pmatrix} 1 & 1 \\ 0 & \mathrm{fl}(1-10^{-17}) \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 2 \\ \mathrm{fl}(1-2\cdot 10^{-17}) \end{pmatrix}$$

A ténylegesen ábrázolt rendszer:

$$\begin{aligned} x + y &= 2 \\ y &= 1 \end{aligned}$$

Megoldása: $y = 1$, $x = 1$ — az elérhető *legjobb* közelítés.

Mi történt?

- A $\ell_{21} = 10^{17}$ óriási szorzó mellett az eredeti 1-es együttható teljesen elnyelődött: $\mathrm{fl}(1 - 10^{17}) = -10^{17}$.
- Ezzel a 2. egyenletből eltűnt minden információ az 1. egyenletről.
- Pivotálással $|\ell_{ik}| \leq 1$, így az elnyelődés nem következik be.

A mátrix nem is volt rosszul kondicionált — az algoritmus volt instabil.

A főelemkiválasztás fajtái

Részleges (oszlop szerinti) főelemkiválasztás

A k. lépésben a k. oszlop k. sorától lefelé a legnagyobb abszolút értékű elemet hozzuk sorcserével a főátlóba. $\Rightarrow$ $|\ell_{ik}| \leq 1$

Költsége elhanyagolható. Ez a gyakorlat alapértelmezése.

Teljes főelemkiválasztás

A teljes maradék részmátrixból választjuk a legnagyobbat (sor- és oszlopcserével). Stabilabb, de drágább, és az ismeretleneket is cserélgetni kell — ritkán éri meg.

Mennyire érzékeny a feladat?

Példa

$$\begin{pmatrix} 1 & 1 \\ 1 & 1{,}0001 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2 \\ 2{,}0001 \end{pmatrix} \quad \Longrightarrow \quad x = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Változtassuk meg a jobb oldalt 0,005%-kal:

$$b = \begin{pmatrix} 2 \\ 2{,}0002 \end{pmatrix} \quad \Longrightarrow \quad x = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

A megoldás 100%-ot változott!

A kondíciószám

Definíció

$$\mathrm{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

Hibabecslés

Ha $A(x + \Delta x) = b + \Delta b$, akkor

$$\frac{\|\Delta x\|}{\|x\|} \leq \mathrm{cond}(A) \cdot \frac{\|\Delta b\|}{\|b\|}$$

A kondíciószám a hibanagyító tényező; $\mathrm{cond}(A) \geq 1$ mindig.

Hány jegyet veszítünk?

Ökölszabály

$$\text{cond}(A) \approx 10^t \implies \text{kb. } t \text{ értékes jegy vész el.}$$

- Az előző példában $\text{cond}(A) \approx 4 \cdot 10^4$ — kb. 4–5 jegy.
- Ha $\text{cond}(A) > 1/\varepsilon_{\text{gép}} \approx 10^{16}$, a mátrix a gép számára gyakorlatilag szinguláris.

Fontos: ez a *feladat* tulajdonsága — a legjobb algoritmus sem segít rajta.

A Hilbert-mátrix

$$h_{ij} = \frac{1}{i+j-1}, \qquad H_3 = \begin{pmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{pmatrix}$$

$H_n x = b$, ahol a pontos megoldás $x = (1, \ldots, 1)$:

n	$\mathrm{cond}(H_n)$	rel. hiba
3	$5{,}2 \cdot 10^{2}$	$8 \cdot 10^{-15}$
5	$4{,}8 \cdot 10^{5}$	$1 \cdot 10^{-12}$
8	$1{,}5 \cdot 10^{10}$	$4 \cdot 10^{-8}$
10	$1{,}6 \cdot 10^{13}$	$2 \cdot 10^{-5}$

10×10-es rendszer, `double` aritmetikával, mégis csak 5 jegy pontos.

Az ötlet

A Gauss-elimináció lépései mátrixszorzásként is leírhatók — és ebből az A mátrix egy felbontása adódik.

Ugyanazon az A mátrixon:

$$\begin{pmatrix} -2 & -1 & 4 \\ 2 & 3 & -1 \\ -4 & -10 & -5 \end{pmatrix} \xrightarrow[\ell_{31}=2]{\ell_{21}=-1} \begin{pmatrix} -2 & -1 & 4 \\ 0 & 2 & 3 \\ 0 & -8 & -13 \end{pmatrix}$$

$$\text{azaz} \quad A = \begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 2 & 0 & 1 \end{pmatrix} \begin{pmatrix} -2 & -1 & 4 \\ 0 & 2 & 3 \\ 0 & -8 & -13 \end{pmatrix}$$

A második lépés

$$\begin{pmatrix} -2 & -1 & 4 \\ 0 & 2 & 3 \\ 0 & -8 & -13 \end{pmatrix} \xrightarrow{\ell_{32}=-4} \begin{pmatrix} -2 & -1 & 4 \\ 0 & 2 & 3 \\ 0 & 0 & -1 \end{pmatrix}$$

Mivel

$$\begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 2 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -4 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 2 & -4 & 1 \end{pmatrix},$$

a szorzók egyszerűen beíródnak az L mátrixba.

Az LU-felbontás

$$A = \begin{pmatrix} -2 & -1 & 4 \\ 2 & 3 & -1 \\ -4 & -10 & -5 \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 2 & -4 & 1 \end{pmatrix}}_{L} \underbrace{\begin{pmatrix} -2 & -1 & 4 \\ 0 & 2 & 3 \\ 0 & 0 & -1 \end{pmatrix}}_{U}$$

$$A = LU$$

- L: alsóháromszög-mátrix, átlójában csupa 1-es
- U: felsőháromszög-mátrix

A megoldás két lépésben

Az eredeti feladat: $Ax = b \implies LUx = b$

$$1.\ Ly = b \qquad 2.\ Ux = y$$

- Mindkét rendszer mátrixa háromszög alakú $\Rightarrow$ egyszerű visszahelyettesítés.
- $Ly = b$: felülről lefelé, $Ux = y$: alulról felfelé.
- Mindkettő $n^2/2$ művelet.

Mire jó ez?

Több jobb oldal, egy mátrix

A felbontás egyszer kell ($n^3/3$); utána minden új b-hez csak n^2 művelet. Gauss-eliminációval minden b-hez újra $n^3/3$ kellene.

A determináns

$$\det(A) = \det(L) \cdot \det(U) = 1 \cdot \det(U) = \prod_{i=1}^{n} u_{ii}$$

A példában: $\det(A) = (-2) \cdot 2 \cdot (-1) = 4$.

Tárigény: A felülírása

Az L átlója csupa 1-es, az alatta lévő helyek pedig U-ban nullák — így egyetlen mátrixban elfér mindkettő:

$$\begin{pmatrix} -2 & -1 & 4 \\ 2 & 3 & -1 \\ -4 & -10 & -5 \end{pmatrix} \to \begin{pmatrix} -2 & -1 & 4 \\ \boxed{-1} & 2 & 3 \\ \boxed{2} & -8 & -13 \end{pmatrix} \to \begin{pmatrix} -2 & -1 & 4 \\ \boxed{-1} & 2 & 3 \\ \boxed{2} & \boxed{-4} & -1 \end{pmatrix}$$

A bekeretezett elemek L-hez, a többi U-hoz tartozik — nincs szükség plusz tárhelyre.

Példa: 4×4-es rendszer

Oldjuk meg LU-felbontással, és adjuk meg $\det(A)$-t!

$$A = \begin{pmatrix} 2 & -1 & 1 & 3 \\ -4 & 0 & 2 & -5 \\ 6 & -1 & -2 & 6 \\ 4 & 2 & -10 & -1 \end{pmatrix}, \qquad b = \begin{pmatrix} -3 \\ -5 \\ 2 \\ 19 \end{pmatrix}$$

$$A \longrightarrow \begin{pmatrix} 2 & -1 & 1 & 3 \\ \boxed{-2} & -2 & 4 & 1 \\ \boxed{3} & \boxed{-1} & -1 & -2 \\ \boxed{2} & \boxed{-2} & \boxed{4} & 3 \end{pmatrix}$$

A két visszahelyettesítés

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ 3 & -1 & 1 & 0 \\ 2 & -2 & 4 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & -1 & 1 & 3 \\ 0 & -2 & 4 & 1 \\ 0 & 0 & -1 & -2 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

$$\underbrace{Ly = b}_{y = (-3,\ -11,\ 0,\ 3)^{\mathsf{T}}} \qquad \underbrace{Ux = y}_{x = (-1,\ 2,\ -2,\ 1)^{\mathsf{T}}}$$

$$\det(A) = 2 \cdot (-2) \cdot (-1) \cdot 3 = 12$$

Ha sorcserére van szükség: PLU

Permutációs mátrix

Az egységmátrix sorainak permutálásával kapjuk. Ha P az i. és j. sort cseréli:

PA: az A sorai cserélődnek, AP: az A oszlopai cserélődnek.

$$A = PLU$$

P: permutációs, L: alsóháromszög (1-es átló), U: felsőháromszög

A megoldás PLU-val

$$A = PLU \qquad \Longrightarrow \qquad LUx = P^{-1}b$$

1. $Ly = P^{-1}b$ 2. $Ux = y$

P ortogonális, ezért $P^{-1} = P^{\mathsf{T}}$ — a gyakorlatban csak egy indexvektort tárolunk, nem az egész mátrixot.

Cholesky-felbontás

Definíció

Az A felbontását $A = LL^T$ alakban, ahol L alsóháromszög-mátrix, Cholesky-felbontásnak nevezzük.

Létezés

Pontosan akkor létezik invertálható L-lel, ha A szimmetrikus és pozitív definit.

L^T-t nem kell külön tárolni, ezért a tár- és műveletigény kb. fele az LU-felbontásénak.

A k-adik lépés

(L-lel felülírva A-t, $k = 1, \ldots, n$:)

$$a_{kk} = \sqrt{a_{kk}}$$
$$a_{ik} = \frac{a_{ik}}{a_{kk}}, \qquad i = k+1, \ldots, n$$
$$a_{ij} = a_{ij} - a_{ik} \cdot a_{jk}, \quad j = k+1, \ldots, n, \; i = j, \ldots, n$$

A gyökvonás miatt kell a pozitív definitség.

Cholesky — példa

$$A = \begin{pmatrix} 9 & -3 & -6 \\ -3 & 5 & 4 \\ -6 & 4 & 9 \end{pmatrix} \to \begin{pmatrix} \boxed{3} & -1 & -2 \\ -1 & 4 & 2 \\ -2 & 2 & 5 \end{pmatrix} \to \begin{pmatrix} 3 & -1 & -2 \\ -1 & \boxed{2} & 1 \\ -2 & 1 & 4 \end{pmatrix}$$

$$A = \underbrace{\begin{pmatrix} 3 & 0 & 0 \\ -1 & 2 & 0 \\ -2 & 1 & 2 \end{pmatrix}}_{L} \underbrace{\begin{pmatrix} 3 & -1 & -2 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{pmatrix}}_{L^{\mathsf{T}}}$$

Miért iterálunk?

- Ha az egyenletrendszer mátrixa túl nagy, akkor a direkt módszer használhatatlanul nagy műveletigényű $\Rightarrow$ közelítő megoldást kell találnunk.
- Iterációs eljárás: elindulunk valahonnan (egy tippelt megoldásból), majd lépésenként egyre közelebb kerülünk a valódi megoldáshoz — de általában pontosan nem érjük el azt.
- Akkor jó egy iteráció, ha az általa megadott sorozat a megoldáshoz konvergál.

Mennyivel olcsóbb?

n	Gauss ($n^3/3$)	1 iterációs lépés ($\approx 2n^2$)
100	$3{,}3 \cdot 10^5$	$2{,}0 \cdot 10^4$
1 000	$3{,}3 \cdot 10^8$	$2{,}0 \cdot 10^6$
10 000	$3{,}3 \cdot 10^{11}$	$2{,}0 \cdot 10^8$

Ha néhány tucat lépés elég, az iteráció nagyságrendekkel gyorsabb.

Ritka mátrixnál még jobb: ott egy lépés csak a nemnulla elemeken fut.

Az iterációs módszerek alakja

Eddigi formánk: $Ax = b$. Az iterációs módszerek általános alakja:

$$x^{(k+1)} = Bx^{(k)} + c$$

azaz valahogyan kapunk egy új tippet ($x^{(k+1)}$) a régi tippből ($x^{(k)}$).

Alapkövetelmény

Ha $x^{(k)}$ már megoldás, akkor $x^{(k+1)}$ is az maradjon: $x = Bx + c$.

Az első megoldási javaslat

Fejezzük ki az x_i változót az i. egyenletből!

Ezt fogjuk Jacobi-iterációnak hívni.

Jacobi — példa

$$\begin{aligned} 5x_1 + x_2 &= 7 \\ x_1 + 2x_2 &= 5 \\ x_2 + 3x_3 &= 2 \end{aligned}$$

Fejezzünk ki minden egyenletből egy-egy változót:

$$x_1^{(k+1)} = -\tfrac{1}{5}x_2^{(k)} + \tfrac{7}{5}, \quad x_2^{(k+1)} = -\tfrac{1}{2}x_1^{(k)} + \tfrac{5}{2}, \quad x_3^{(k+1)} = -\tfrac{1}{3}x_2^{(k)} + \tfrac{2}{3}$$

A jobb oldalon mindenhol a régi (k-adik) értékek állnak.

Mátrix alakban

$$\begin{pmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ x_3^{(k+1)} \end{pmatrix} = \begin{pmatrix} 0 & -1/5 & 0 \\ -1/2 & 0 & 0 \\ 0 & -1/3 & 0 \end{pmatrix} \begin{pmatrix} x_1^{(k)} \\ x_2^{(k)} \\ x_3^{(k)} \end{pmatrix} + \begin{pmatrix} 7/5 \\ 5/2 \\ 2/3 \end{pmatrix}$$

Indítsuk el az iterációt az $x^{(0)} = (1 \;\; 1 \;\; 1)^\mathsf{T}$ kezdővektorból:

$$x^{(1)} = \begin{pmatrix} 6/5 \\ 2 \\ 1/3 \end{pmatrix}$$

A további lépések

$$x^{(2)} = \begin{pmatrix} 1 \\ 1{,}9 \\ 0 \end{pmatrix} \qquad x^{(3)} = \begin{pmatrix} 1{,}02 \\ 2 \\ 0{,}03 \end{pmatrix} \qquad x^{(4)} = \begin{pmatrix} 1 \\ 1{,}99 \\ 0 \end{pmatrix}$$

A pontos megoldás: $x^* = (1,\ 2,\ 0)^{\mathsf{T}}$

Látható, hogy a sorozat oszcillálva közelít: a páros és a páratlan indexű tagok két oldalról tartanak a megoldáshoz.

A Jacobi-iteráció mátrixokkal

Az i. változót az i. egyenletből kifejezni azt jelenti, hogy a főátlón kívül mindent átviszünk a jobb oldalra. Ha D az A főátlójából képzett diagonális mátrix:

$$Dx = b - (A - D)x$$

A Jacobi-iteráció

$$x^{(k+1)} = \underbrace{-D^{-1}(A - D)}_{B_J} x^{(k)} + \underbrace{D^{-1}b}_{c}$$

Feltétel: $a_{ii} \neq 0$ minden i-re (különben D nem invertálható).

Egy jobb ötlet

Tegyük föl, hogy az új x értékeket nem egyszerre (mátrixművelettel), hanem egyesével számoljuk: először frissítjük x_1-et, majd x_2-t, stb.

Ekkor x_2 frissítéséhez már x_1 új értékét használhatjuk, nem a régit!

Ez a Gauss–Seidel-iteráció.

Gauss–Seidel — ugyanazon a példán

$$
\begin{aligned}
x_1^{(k+1)} &= -\tfrac{1}{5}x_2^{(k)} + \tfrac{7}{5} \\
x_2^{(k+1)} &= -\tfrac{1}{2}x_1^{(k+1)} + \tfrac{5}{2} \\
x_3^{(k+1)} &= -\tfrac{1}{3}x_2^{(k+1)} + \tfrac{2}{3}
\end{aligned}
$$

A különbség egyetlen felső index: a már kiszámolt $(k+1)$-edik értékeket azonnal felhasználjuk.

Az első két lépés

$x^{(0)} = (1,1,1)^{\mathsf{T}}$-ból indulva:

1. iteráció

$$x_1^{(1)} = -\tfrac{1}{5} \cdot 1 + \tfrac{7}{5} = 1{,}2$$
$$x_2^{(1)} = -\tfrac{1}{2} \cdot 1{,}2 + \tfrac{5}{2} = 1{,}9$$
$$x_3^{(1)} = -\tfrac{1}{3} \cdot 1{,}9 + \tfrac{2}{3} = 0{,}03$$

2. iteráció

$$x_1^{(2)} = -\tfrac{1}{5} \cdot 1{,}9 + \tfrac{7}{5} = 1{,}02$$
$$x_2^{(2)} = -\tfrac{1}{2} \cdot 1{,}02 + \tfrac{5}{2} = 1{,}99$$
$$x_3^{(2)} = -\tfrac{1}{3} \cdot 1{,}99 + \tfrac{2}{3} = 0{,}003$$

Két lépés alatt már 3 tizedesjegy pontosság.

Mátrix alakban

Bontsuk fel A-t: $A = L + D + U$, ahol L a főátló alatti, U a főátló feletti rész.

Jacobi

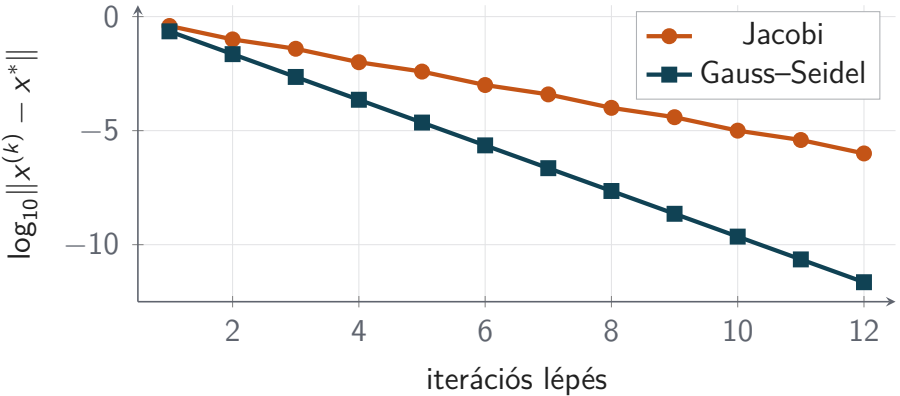
$$B_J = -D^{-1}(L + U)$$

Gauss–Seidel

$$B_{GS} = -(D + L)^{-1}U$$

A $(D + L)^{-1}$-t sosem számoljuk ki: a gyakorlatban egyszerűen soronként haladva, helyben frissítjük a vektort.

A két módszer összevetése



Gauss–Seidel lépésenként egy teljes tizedesjegyet nyer, Jacobi csak felet.

Mikor konvergál az iteráció?

Definíció

Egy iterációs sorozat globálisan konvergens, ha bármely indulóvektorral ugyanazt a megoldást kapjuk.

Definíció: szigorú diagonális dominancia

Az A mátrix szigorúan diagonálisan domináns, ha minden $i = 1, \ldots, n$-re

$$\sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}| \; < \; |a_{ii}|$$

Diagonális dominancia — példa

„A sor abszolútérték-összege (a főátlóbeli elem nélkül) legyen kisebb a főátlóbeli elem abszolút értékénél.”

$$A = \begin{pmatrix} -6 & 4 & 1 \\ 0 & 15 & 2 \\ 10 & 10 & -21 \end{pmatrix}$$

$$|-6| = 6 > 5 = |4| + |1|$$
$$|15| = 15 > 2 = |0| + |2|$$
$$|-21| = 21 > 20 = |10| + |10|$$

Szigorúan diagonálisan domináns. ✓

Elegendő feltétel

Tétel

A Jacobi- és a Gauss–Seidel-iteráció is konvergens, ha a hozzá tartozó egyenletrendszer mátrixa szigorúan diagonálisan domináns.

Ez elegendő, de nem szükséges feltétel:

lehet olyan, nem diagonálisan domináns mátrixú rendszer, amelyre valamelyik módszer mégis konvergál.

A pontos feltétel: a spektrálsugár

Definíció

A B mátrix spektrálsugara a sajátértékei abszolút értékének maximuma: $\rho(B) = \max_i |\lambda_i|$.

Tétel (szükséges és elegendő)

Az $x^{(k+1)} = Bx^{(k)} + c$ iteráció pontosan akkor konvergens minden kezdővektorra, ha

$$\rho(B) < 1$$

Minél kisebb $\rho(B)$, annál gyorsabb a konvergencia.

A példánkra

$$\rho(B_J) = 0{,}316 \qquad \rho(B_{GS}) = 0{,}100$$

Itt éppen $\rho(B_{GS}) = \rho(B_J)^2$ —

ezért nyer a Gauss–Seidel lépésenként kétszer annyi jegyet.

Ez a kapcsolat bizonyos mátrixosztályokra (pl. tridiagonális) tételként is igaz, de általában nem.

Figyelem: a Gauss–Seidel nem mindig jobb!

$$A = \begin{pmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{pmatrix}$$ (nem diagonálisan domináns)

Jacobi

$\rho(B_J) = 0$

3 lépésben *pontosan* megoldja

Gauss–Seidel

$\rho(B_{GS}) = 2$

divergál: a hiba duplázódik

Van olyan mátrix is, ahol fordítva igaz — a $\rho(B) < 1$ feltételt kell ellenőrizni.

Mikor álljunk le?

Honnan tudjuk, hogy elég közel értünk? A valódi megoldást nem ismerjük, így az $\|x^{(k)} - x^*\|$ hibát sem.

A gyakorlatban használt leállási feltételek

- két egymást követő lépés eltérése: $\|x^{(k+1)} - x^{(k)}\| < \varepsilon$
- a reziduum: $\|Ax^{(k)} - b\| < \varepsilon$
- maximális lépésszám (biztonsági fék)

Mindkét feltételt érdemes relatív alakban használni, pl. $\|Ax^{(k)} - b\| < \varepsilon\|b\|$.

Vektornormák

Két vektor közelségének mérésére normát használunk:

$$\|x\|_2 = \sqrt{\sum_i x_i^2}, \qquad \|x\|_1 = \sum_i |x_i|, \qquad \|x\|_\infty = \max_i |x_i|$$

- A 2-es norma a vektor szokásos hossza (`norm(v)` MATLAB-ban).
- Véges dimenzióban mindegyik norma ekvivalens: ha az egyikben kicsi a hiba, a többiben is az.

Tehát ha $\|x^{(k)} - x^*\|$ már elég kicsi, mondhatjuk, hogy konvergáltunk.

Gyorsítás: az SOR-módszer

A Gauss–Seidel lépést „túllőve” gyorsíthatunk. Ha x_i^{GS} a Gauss–Seidel-lépés eredménye:

SOR (szukcesszív túlrelaxálás)

$$x_i^{(k+1)} = (1-\omega)\, x_i^{(k)} + \omega\, x_i^{GS}$$

- $\omega = 1$: éppen a Gauss–Seidel-iteráció
- $1 < \omega < 2$: túlrelaxálás — jól megválasztott ω-val nagyságrendi gyorsulás érhető el
- konvergenciához szükséges: $0 < \omega < 2$

Melyiket válasszuk?

	direkt	**iteratív**
kis, sűrű mátrix	igen	nem
nagy, ritka mátrix	nem	igen
sok jobb oldal, egy A	LU	—
szimmetrikus, poz. definit	Cholesky	CG
pontos megoldás kell	igen	csak közelítés
memória	$\mathcal{O}(n^2)$	$\mathcal{O}(\text{nemnulla})$

A modern gyakorlatban nagy rendszerekre a Krylov-módszerek (CG, GMRES) + előkondicionálás a standard.

A gyakorlatban

MATLAB

`x = A\b` — automatikusan választ: háromszög, szimmetrikus (Cholesky), általános (PLU)...

`[L,U,P] = lu(A)`, `chol(A)`, `cond(A)`

Python (SciPy)

`scipy.linalg.solve(A, b)`, `lu_factor` / `lu_solve`

`scipy.sparse.linalg.cg`, `gmres` — ritka mátrixokra

Soha ne számoljunk `inv(A)*b`-t — lassabb és pontatlanabb.

Összefoglalás — direkt módszerek

- Gauss-elimináció: $n^3/3$ művelet; a visszahelyettesítés $n^2/2$.
- Pivotálás nélkül az elimináció numerikusan instabil lehet — a részleges főelemkiválasztás szinte ingyen van.
- A kondíciószám a feladat érzékenységét méri: $\mathrm{cond}(A) \approx 10^t$ esetén t jegy vész el.
- LU: ugyanaz az ár, de több jobb oldalhoz újra felhasználható; $\det(A) = \prod u_{ii}$.
- Cholesky: szimmetrikus, pozitív definit mátrixra fele annyi munka.

Összefoglalás — iteratív módszerek

- Egy lépés csak $\mathcal{O}(n^2)$ — nagy, ritka mátrixnál ez dönt.
- Jacobi: a régi értékekből számol; Gauss–Seidel: azonnal használja az újakat.
- Konvergencia pontosan akkor, ha $\rho(B) < 1$.
- A szigorú diagonális dominancia elegendő (de nem szükséges) feltétel mindkét módszerhez.
- Leállás: a lépésköz vagy a reziduum normája, relatív alakban.

Irodalom

[1] Baran Ágnes: *Numerikus matematika* — Lineáris egyenletrendszerek fejezet.

[2] *Iterációk: Jacobi, Gauss–Seidel* — SZTE előadásvázlat (Gelle Kitti nyomán).

[3] *Numerikus módszerek* — Kelet-Magyarországi Informatikai Tananyag Tárház, Debreceni Egyetem, `gyires.inf.unideb.hu/KMITT`

[4] Stoyan G., Takó G.: *Numerikus módszerek I.* Typotex.

[5] G. H. Golub, C. F. Van Loan: *Matrix Computations*. 4. kiadás, Johns Hopkins University Press, 2013.

[6] T. A. Driscoll, R. J. Braun: *Fundamentals of Numerical Computation*. SIAM, 2017.